

The Process Model

Definition

In the **process model**, all runnable software in the computer is organized as a number of **sequential processes**. A process is an instance of a program in execution, together with the current values of its program counter, registers, and variables.

1. Basic Idea

Main Concept

Conceptually, each process has its own **virtual CPU**. In reality, the real CPU switches rapidly from one process to another, but it is easier to understand the system by imagining many processes running in parallel.

Multiprogramming

This rapid switching among processes is called **multiprogramming**. It creates the illusion that multiple processes are progressing at the same time.

2. Processes and Virtual CPU

Virtual CPU

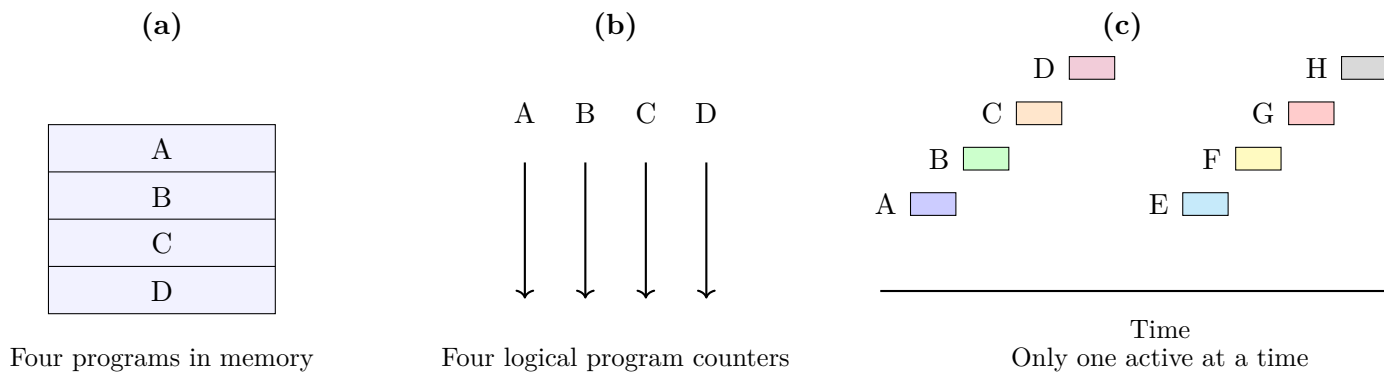
Each process appears to have its own CPU and its own flow of control. This is only a conceptual model, but it helps explain how the operating system manages many activities together.

Reality

Actually:

- there is only one real CPU in the simple model,
- only one real program counter exists,
- when one process runs, its logical program counter is loaded into the real CPU,
- when it stops, that value is saved again.

2.1 Conceptual Process Model



3. Single CPU Assumption

Simplifying Assumption

To understand the basic idea, we often assume that the system has only **one CPU**. In reality, many modern systems have multicore processors, but the single-CPU model is easier to study first.

Important Note

Even if a computer has two or more cores, each individual core can still execute only one process at a time.

4. Timing and Process Execution

Nonuniform Execution

Because the CPU switches among many processes, a process does not run at a perfectly steady rate. Its progress may vary from one execution to another.

Programming Rule

Processes should not be written with fixed assumptions about timing. A process may be interrupted and resumed later, so timing based only on CPU loops is unreliable.

Audio-Video Example

Suppose an audio process wants to wait a little before starting sound to match a video. If it simply runs a loop 10,000 times as a delay, the CPU may switch to another process during that loop. Then the sound may start too late and become out of sync with the video.

Real-Time Requirement

If a process must perform an action within a specific number of milliseconds, special real-time measures are needed to guarantee correct timing.

5. Process vs Program

Key Difference

The difference between a **process** and a **program** is very important:

- a **program** is passive, stored on disk,
- a **process** is active, currently executing.

Short Meaning

A program is just code. A process is the actual activity of running that code.

6. Cake Analogy

Cake Example

Consider a person baking a cake:

- the recipe is the **program**,
- the baker is the **processor**,
- the ingredients are the **input data**,
- the baking activity is the **process**.

Interruption Example

If the baker stops making the cake to give first aid to someone, the current state of the cake-making activity must be remembered. Later, the baker returns and continues from the same point. This is similar to the CPU switching from one process to another and then back again.

7. What a Process Has

Components of a Process

A process has:

- a program,
- input data,
- output,
- a current state,
- the current program counter,
- register values,
- variable values.

8. Scheduling

Scheduling

Since one processor may be shared among several processes, the operating system uses a **scheduling algorithm** to decide when to stop one process and run another.

Purpose

Scheduling helps the system share the CPU fairly and efficiently among many processes.

9. One Program, Multiple Processes

Important Fact

If the same program runs twice, it counts as **two different processes**.

Example

A user may:

- open the same word processor twice,
- print two files at the same time,
- or run the same command in multiple windows.

Even if the program code is the same, these are separate processes.

Code Sharing

The operating system may keep only one copy of the program code in memory and let both processes share it, but conceptually they are still two different processes.

10. Summary Points

Quick Review

- A process is an executing instance of a program.
- Each process is viewed as if it has its own virtual CPU.
- CPU switching among processes is called multiprogramming.
- Timing of process execution is not uniform.
- A process is active, but a program is passive.
- Running the same program twice creates two separate processes.